#BLACKBOX CTF — Solver Write-up

Challenge

Name: The Broken License

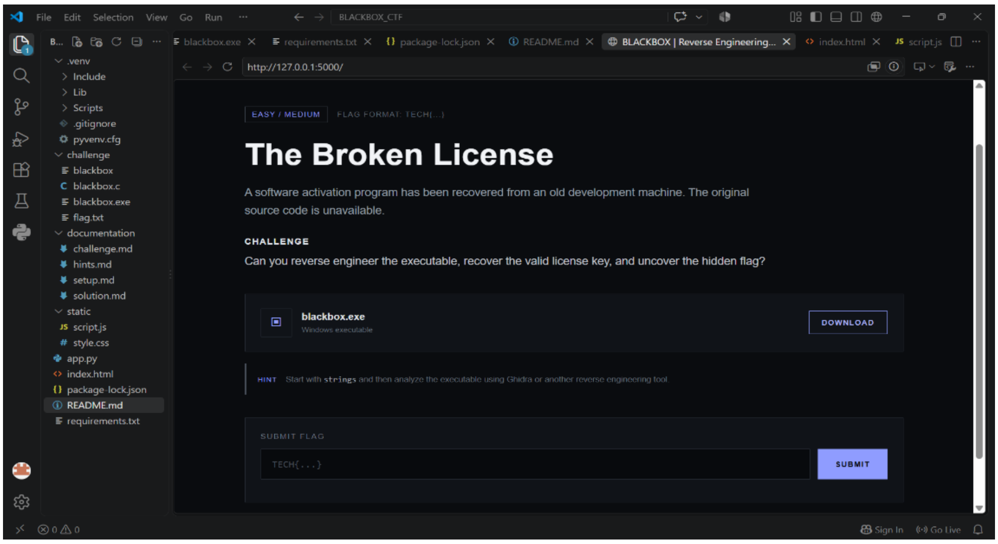
Category: Reverse Engineering

Flag: `TECH{reverse_the_blackbox}`

1. Reconnaissance

I was provided with a Windows executable named `blackbox.exe`.

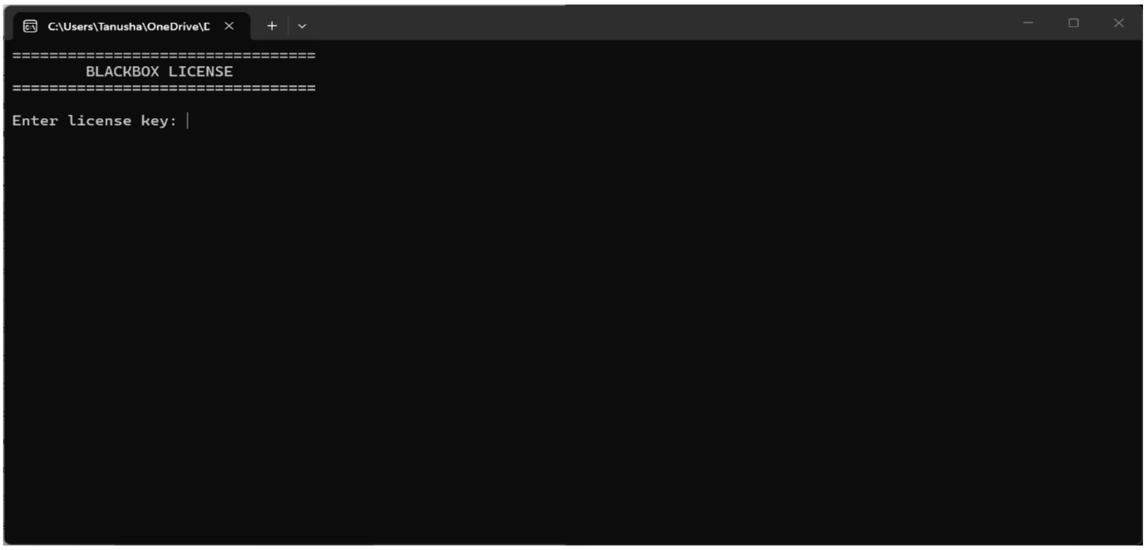
I first executed it using:

```powershell
.\blackbox.exehe program asked for a license key.

**Screenshot 1:**




---

**2. Testing**

I entered an incorrect value such as:
```

TEST

The program rejected the input, which indicated that a license validation mechanism was present.

Screenshot 2:



---

3. Reverse Engineering

I imported blackbox.exe into **Ghidra** and performed static analysis.

I first inspected the strings and searched for messages such as:

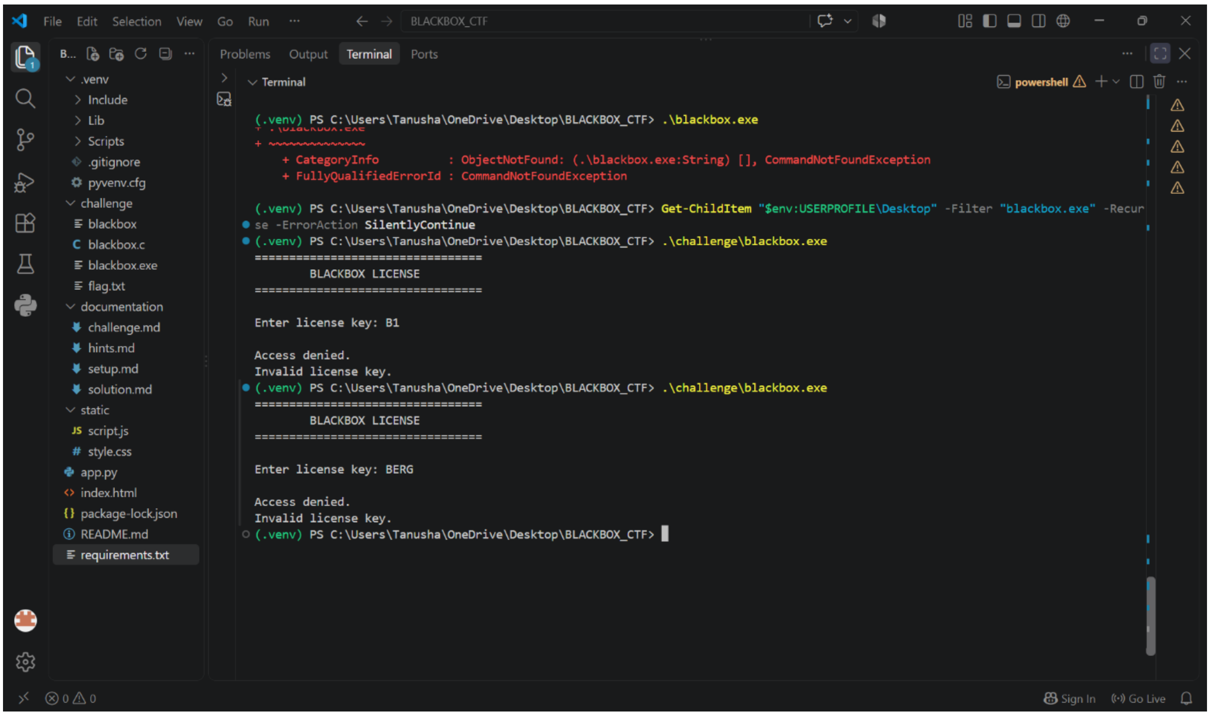
Enter license key

Access granted

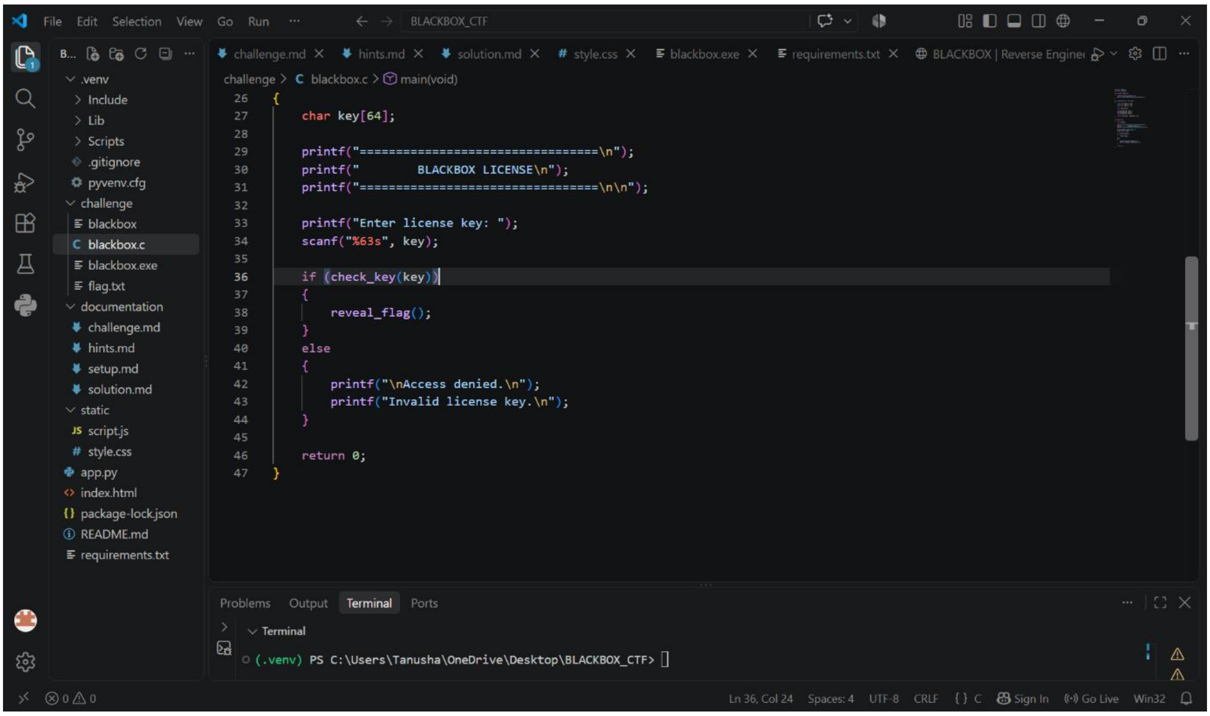
Access denied

I then followed the references to locate the license-validation logic.

Screenshot 3:



Screenshot 4:



---

4. Recovering the Key

The validation logic contained three separate key fragments:

B1

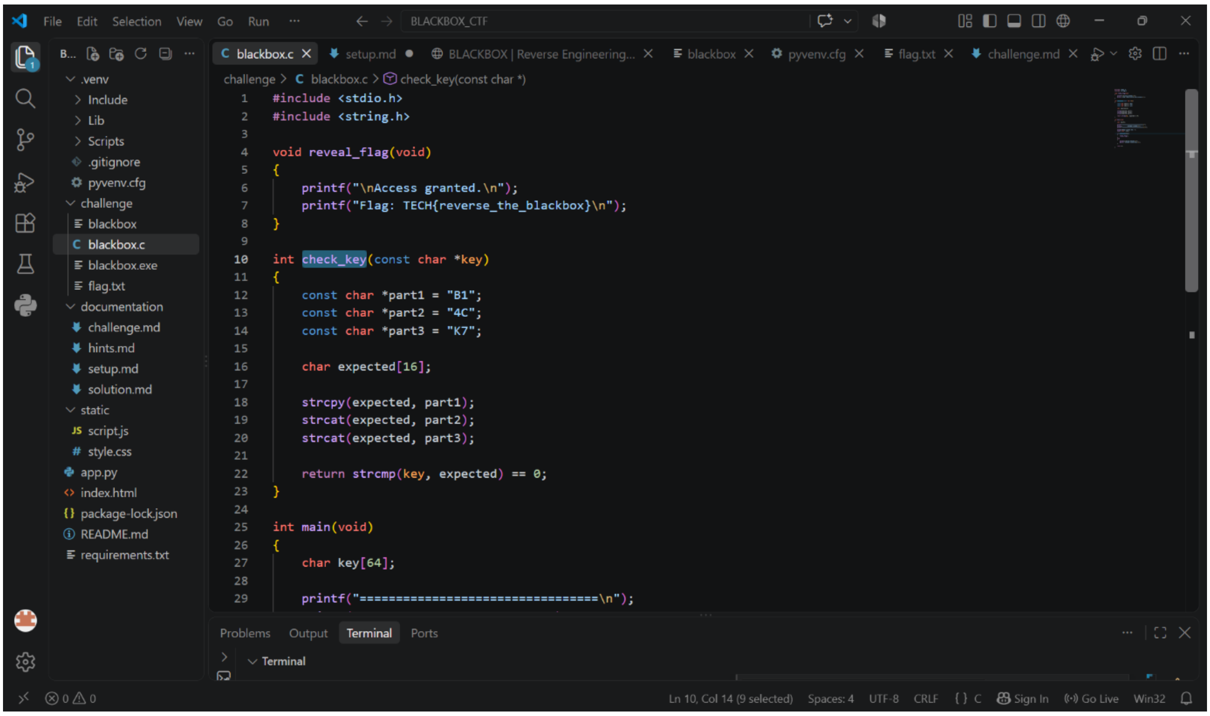
4C

K7

Combining them in the required order gave:

B14CK7

Screenshot 5:



---

5. Verification

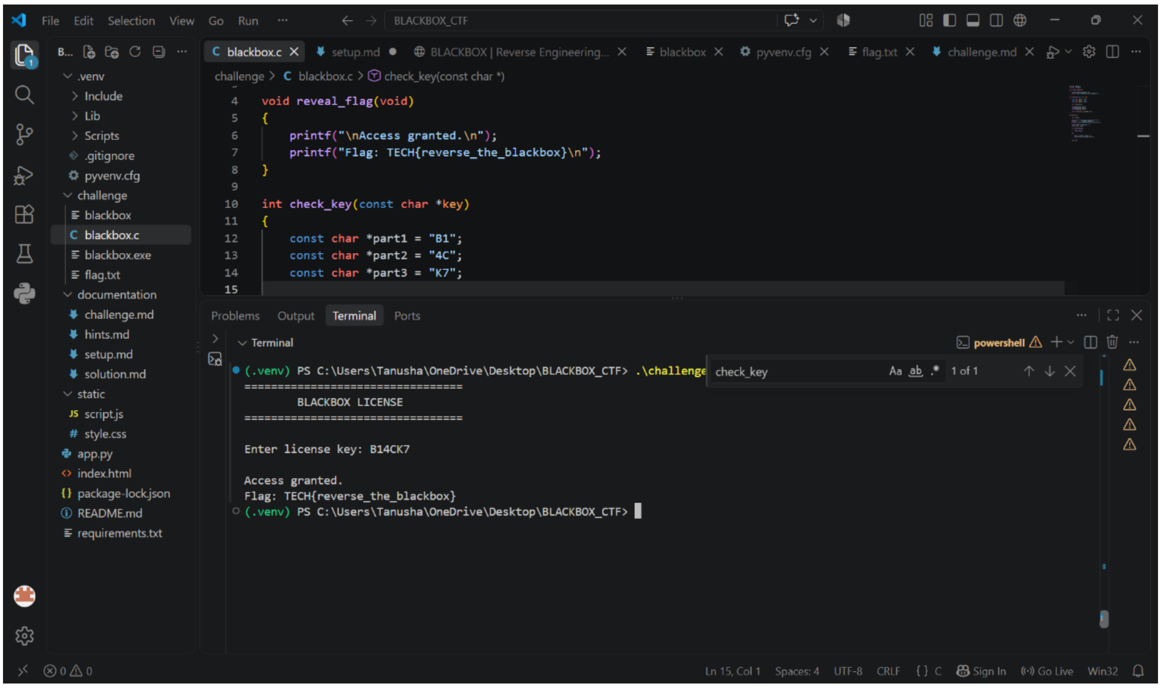
I ran the executable again and entered:

B14CK7

The program accepted the key and revealed:

TECH{reverse_the_blackbox}

Screenshot 6:

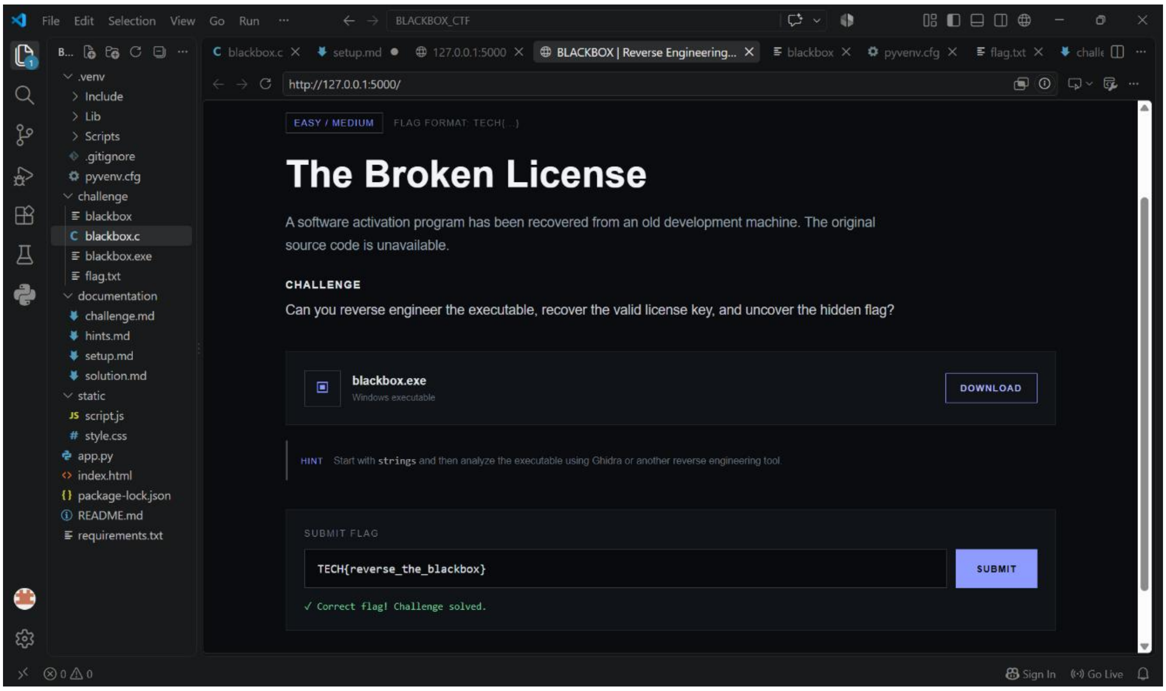


---

6. Flag Submission

I entered the recovered flag into the CTF web interface and received a successful submission message.

Screenshot 7:



Tools Used

- PowerShell
- Ghidra
- GCC/MinGW

Methodology

Run binary

→ Test input

→ Analyze with Ghidra

→ Find validation logic

→ Recover key

→ Verify key

→ Obtain flag

→ Submit flag

Conclusion

The challenge demonstrated how static analysis can be used to understand an executable's validation logic and recover information hidden inside a compiled program.